

Should all CRUD logic (add/search/update/delete) for contracts, salaries, match records, and training sessions live directly inside Main.java, or should each entity get its own List manager class like ContractList and SalaryList?

Weighed architectural patterns and reconciled with existing codebase conventions >

Your codebase already answers this empirically — every entity (`Contract`, `Salary`, `MatchRecord`, `TrainingSession`, `Player`) already has its own dedicated `*List` manager class, and `Main` is already thin (it just owns instances and routes menu choices). So this isn't really "should we," it's "is the existing pattern correct" — and yes, here's why.

Option A: All CRUD inside `Main.java`

What it would look like:

```
java

public class Main {
    private ArrayList<Contract> contracts;
    private ArrayList<Salary> salaries;
    private ArrayList<MatchRecord> matchRecords;
    private ArrayList<TrainingSession> sessions;
    // ... plus addContract(), searchContract(), updateContract(), deleteContract(),
    //      addSalary(), searchSalary(), ... × 4 entities × 4 operations = 16+ methods
}
```

Cons (these are severe, not minor):

- `Main` becomes a "God class" — single file owning data storage *and* business logic *and* menu routing *and* validation for every entity. Any change to contract logic risks touching a file that also controls salaries, training, and the program's entry point.
- No encapsulation — `contracts` would need to be exposed (or `Main` becomes the only place that can ever touch it), making it impossible to reuse contract logic elsewhere without dragging `Main` along.
- Testing is harder — you can't unit-test "add a contract" without spinning up the whole `Main` object and its menu-driven flow.
- This is the exact opposite of what your `SalaryList.addNewSalaryRecord(sc, playerList)` already demonstrates — it needs to *cross-reference* `PlayerList`, which only works cleanly because each manager is a separate, addressable object you can pass around.

Pros: Genuinely none for a project at this scale — maybe marginally less boilerplate for a 10-line throwaway script, but not here.

Option B: Each entity gets its own List manager (current design)

Pros:

- **Single Responsibility Principle** — `ContractList` only knows about contracts; `SalaryList` only knows about salaries (but is allowed to ask `PlayerList` questions via its own public interface — composition, not ownership).
- `Main` stays a thin coordinator:

```
java
case 4:
    contractList.manageContracts(scanner, playerList);
    break;
```

This is the whole responsibility `Main` needs for contracts — one line. That's a strong sign of good separation.

- Each manager class can independently validate, search, and mutate its own `ArrayList` without any other class needing to know about its internals (`contractCollection`, `salaryRecords`, etc. are all `private`).
- Matches exactly how you're already cross-validating: `Contract` needs `Player` to exist (`playerList.findPlayerById(...)`), `Salary` needs `Player` to exist *and* needs to call `player.calculateMonthlySalary()`. Both only work because `PlayerList` and `Player` are decoupled, addressable units that get passed in, not globals baked into `Main`.

Cons:

- More files (5 entities → 5 manager classes) — but this is a deliberate cost for maintainability, not bloat. Each file is small and does one thing.
- Slight repetition across managers (`readMenuNumber` -style helpers are duplicated in `ContractList`, `MatchRecordList`, `SalaryList`, `TrainingSessionList` — each writes its own near-identical menu-reading loop). That's a real, mild violation of DRY worth noting, but it's a much smaller cost than collapsing everything into `Main`.

Recommendation

Keep the per-entity manager pattern you already have. It's the correct architecture for this scale of system, and it's the reason features like contract-existence checks before adding a salary, or shirt-number uniqueness checks in `PlayerList`, are even feasible to write cleanly. The one mild improvement I'd suggest — if you want to tighten things further — is extracting the repeated "read an integer between min and max" menu logic into a shared utility (could live in `ClubEntity` or a small static helper), since right now it's copy-pasted near-verbatim across four classes. Want me to show what that refactor would look like?